

Contents

Week 2 — Foundations: Agent Reasoning & Planning	1
Learning objectives	1
1. Chain-of-Thought and its variants	1
2. ReAct — synergizing reasoning and acting	2
3. Reflexion — verbal reinforcement learning	3
4. Tree-of-Thoughts and Graph-of-Thoughts	4
5. Planning: task decomposition, hierarchical planning, and world models	6
6. In-class demo (instructor-led)	7
7. Discussion questions for this session	7
Further reading	7

Week 2 — Foundations: Agent Reasoning & Planning

CO5151 · Advanced Agentic Artificial Intelligence Learning outcomes touched: L.O.1.1, L.O.1.2 · Feeds directly into: S1-1 (Advanced Reasoning & Planning), S1-6 (Self-improvement & Reflection) · Duration: 150 minutes

Schedule note: the real HK261 timetable gives only 13 sessions, not 15. This content and the **Week 1 notes** are both taught in one combined, flipped-classroom session — real Week 1, Wed 9 Sep 2026, not a separate second meeting. Assign both documents as pre-reading before that class.

Learning objectives

By the end of this session, students can:

1. Explain the mechanism (not just the name) of Chain-of-Thought, self-consistency, ReAct, Reflexion, Tree-of-Thoughts, and Graph-of-Thoughts, and reproduce a worked trace for each.
2. Place each technique on a common cost/accuracy trade-off axis and state the assumption each one makes about the availability of a reward or evaluation signal.
3. Distinguish “reasoning” from “planning” as formalized in the classical AI sense, and explain the core objection to the claim that LLMs can plan on their own.
4. Choose, for a given project task, which reasoning/planning paradigm is appropriate and justify the choice against at least one alternative.

1. Chain-of-Thought and its variants

1.1 The core observation

Wei et al. (NeurIPS 2022) showed that prompting a sufficiently large language model to produce intermediate reasoning steps before its final answer — rather than answering directly — substantially improves accuracy on multi-step arithmetic, symbolic, and commonsense reasoning tasks. The mechanism is simple to state and easy to underestimate: the model’s own previously-generated tokens become part of its context for generating the next token, so writing out “step 1, step 2, step 3” effectively gives the model a scratchpad, decomposing a hard single-shot prediction into several easier ones conditioned on each other.

Direct prompting:

Q: Roger has 5 tennis balls. He buys 2 cans of 3 balls each. How many does he have now?

A: 11 ← often wrong for weaker models, no visible working

Chain-of-Thought prompting:

Q: Roger has 5 tennis balls. He buys 2 cans of 3 balls each. How many does he have now?

A: Roger started with 5 balls. 2 cans of 3 balls each is $2 \times 3 = 6$ balls.

$5 + 6 = 11$. The answer is 11.

Two important nuances:

- **CoT is a capability that must already exist, elicited, not created.** The original paper shows the effect emerges only at sufficient model scale (an “emergent ability”) — CoT prompting a small model with weak reasoning does not help and can even hurt (it adds tokens the model cannot use coherently). This is why “just add ‘let’s think step by step’” is not a universal fix (see zero-shot CoT below) — it surfaces a capability, it does not install one.
- **Explanations are not guaranteed to be faithful.** The generated reasoning steps are what the model used to arrive at the tokens that follow *only insofar as autoregressive generation makes them part of the context* — there is no guarantee the stated “steps” reflect the actual computation, and models have been shown to produce a plausible-looking chain that doesn’t match the true decision path. Treat CoT output as an artifact that *helps* the model, not as a transparent log of its reasoning — this matters directly for Reflexion and process-reward methods later in this note.

1.2 Zero-shot CoT and self-consistency

Zero-shot CoT (Kojima et al., 2022) showed that simply appending “Let’s think step by step” to a prompt — with *no* worked examples at all — recovers much of the benefit of the original few-shot CoT prompting, which required hand-written exemplar chains. This separates two contributions that are easy to conflate: (a) *reasoning stepwise* helps, and (b) *few-shot exemplars* help — zero-shot CoT shows (a) alone accounts for a large share of the gain.

Self-consistency (Wang et al., ICLR 2023) treats CoT as a *stochastic* process: sample multiple independent reasoning chains (via temperature > 0), then take a majority vote over their final answers, discarding the individual chains.

```
def self_consistency(question, model, n_samples=10):
    answers = []
    for _ in range(n_samples):
        chain = model.generate(cot_prompt(question), temperature=0.7)
        answers.append(extract_final_answer(chain))
    return majority_vote(answers)
```

The insight: individual chains can go wrong in different, largely uncorrelated ways, so voting recovers a signal that a single deterministic chain would miss. The cost is explicit and easy to state: `n_samples` times the tokens and latency of a single chain, for a typically sub-linear accuracy gain — this is your first concrete example of the **test-time compute** trade-off that recurs all semester (ToT, LATS, and reasoning-RL models like o1/DeepSeek-R1 are all, at some level, ways of spending more inference-time compute for better answers).

2. ReAct — synergizing reasoning and acting

2.1 Mechanism

CoT reasons *about* a static context; it cannot acquire new information mid-reasoning. ReAct (Yao et al., ICLR 2023) interleaves free-text reasoning (“thoughts”) with concrete actions against an external environment (e.g., a

search API), feeding each action’s observation back into context before the next thought:

Thought 1: I need to find out who directed the film that won Best Picture in 2023.

Action 1: Search["2023 Best Picture Oscar winner"]

Observation 1: "Everything Everywhere All at Once" won Best Picture at the 95th Academy Awards.

Thought 2: Now I need to find the director(s) of that film.

Action 2: Search["Everything Everywhere All at Once director"]

Observation 2: Directed by Daniel Kwan and Daniel Scheinert.

Thought 3: I have the answer.

Action 3: Finish["Daniel Kwan and Daniel Scheinert"]

This is precisely the control loop from Week 1 §2.2, with decision in that pseudocode split into an explicit Thought (why) and Action (what) pair. The thought is not executed by anything — its only function is to condition the action on an explicit intermediate justification, which the paper shows reduces hallucinated or off-task actions compared to acting without an explicit reasoning step, and reduces reasoning drift compared to CoT without grounding in real observations.

2.2 What ReAct assumes

Naming assumptions is how you turn a mechanism into a critique (see Week 1 §3.3). ReAct’s mechanism implicitly assumes:

- **Actions are cheap and fast.** Each action-observation round trip costs one more LLM call’s worth of latency plus whatever the tool itself costs. A search API call is cheap; a paid transaction, a destructive filesystem operation, or a multi-minute simulation is not — the paper’s benchmarks (HotpotQA, FEVER, ALFWorld, WebShop) all have cheap, fast, reversible actions. Session 3’s discussion of tool *irreversibility* is the direct sequel to this gap.
- **The action space is small and well-specified.** A handful of named actions with clear semantics (Search, Lookup, Finish). Real agents (S1-2, and Week 3) often expose dozens to hundreds of tools — the tool-*selection* problem barely exists in ReAct’s original setting.
- **Observations are immediately truthful and complete.** No adversarial or malicious content in tool results — an assumption that Seminar 2’s security topics (S2-3, S2-4) directly attack: if an observation can contain attacker-controlled text, “feed the observation back into context” is exactly the mechanism indirect prompt injection exploits.

2.3 Where does the gain come from?

A recurring, healthily skeptical question when reading any reasoning-augmentation paper: is the improvement coming from the technique’s structure, or simply from spending more tokens / more forward passes? For ReAct specifically, the paper’s own ablations show that acting-only (no thoughts) and reasoning-only (no actions, i.e., plain CoT) both underperform the combination — supporting a genuine structural claim, not just a compute-budget artifact. You should expect to perform (or demand) this same ablation check for every technique in this course, including the ones below.

3. Reflexion — verbal reinforcement learning

3.1 Mechanism

Reflexion (Shinn et al., NeurIPS 2023) adds a second phase after a ReAct-style attempt fails: the agent reflects in natural language on *why* it failed, and stores that reflection in an episodic memory buffer that is included in

context on the *next* attempt at the same or a similar task.

```
def reflexion(task, model, evaluator, max_trials=4):
    memory = [] # episodic memory of past reflections
    for trial in range(max_trials):
        trajectory = react_loop(task, model, context_extra=memory)
        outcome = evaluator(trajectory) # success/failure signal – see below
        if outcome.success:
            return trajectory
        reflection = model.generate(reflect_prompt(trajectory, outcome))
        memory.append(reflection) # persists to the next trial
    return trajectory # last attempt, possibly still failing
```

The name “verbal reinforcement learning” is precise in one sense and misleading in another: like RL, the agent improves across attempts at the *same* task using a feedback signal — but no weights are updated; the “learning” is entirely the accumulation of natural-language reflections in a growing context. This is why it belongs in Week 2 rather than in S2-6 (RL for Agents), which covers actual gradient-based training.

3.2 The critical dependency: where does **evaluator** come from?

Reflexion’s mechanism is only as good as its feedback signal, and this is the single most important thing to interrogate in the paper and in any system that claims “self-improvement.” Three cases, in decreasing order of how much you should trust the reported results:

1. **External, ground-truth evaluator** (e.g., unit tests pass/fail for a coding task, or an environment’s built-in success signal). Strong, reliable signal — Reflexion’s coding-task results fall here.
2. **External, heuristic evaluator** (e.g., an LLM judge scoring the trajectory). Weaker — inherits whatever biases the judge model has (see §3.3 in Week 1, and S1-8’s discussion of LLM-as-judge).
3. **The same model reflecting on its own output with no external signal at all.** This is the case directly investigated — and shown to be unreliable — by Huang et al. (“Large Language Models Cannot Self-Correct Reasoning Yet,” ICLR 2024), assigned in S1-6: when there is no external ground truth, models cannot reliably tell correct answers from incorrect ones well enough to *usefully* revise them, and self-correction attempts can even *decrease* accuracy by talking the model out of a correct answer.

The rule of thumb for this course: whenever a paper or your own project claims “the agent reflects and improves,” ask immediately what case (1)–(3) above the feedback signal falls into. Case 1 is a real result. Case 3 dressed up as case 1 is the most common inflated claim you will encounter — and it is directly relevant to your project’s R6 security analysis and R5 evaluation requirements: an agent that “self-verifies” its own security posture with no external check has verified nothing.

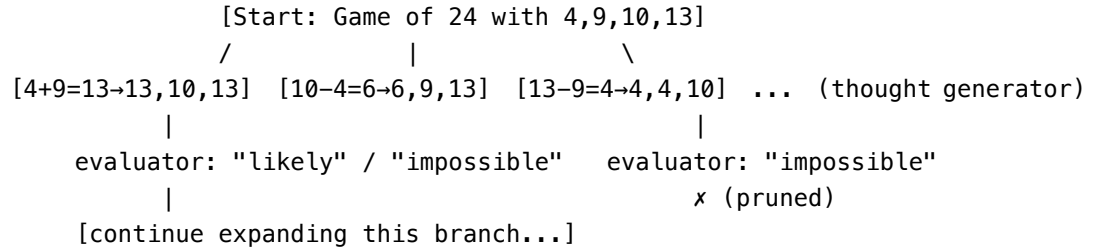
4. Tree-of-Thoughts and Graph-of-Thoughts

4.1 From a chain to a tree

CoT and ReAct both commit to a single linear sequence of steps — once a thought is generated, the model doesn’t reconsider it. Tree-of-Thoughts (Yao et al., NeurIPS 2023) instead treats reasoning as a **search problem** over a tree of partial solutions (“thoughts” as tree nodes), with four components:

- a **thought generator** that proposes several candidate next steps from the current state,
- a **state evaluator** that scores or compares candidate partial solutions,

- a **search algorithm** (breadth-first or depth-first search in the original paper) that decides which branches to expand and which to prune,
- a **stopping rule**.



Graph-of-Thoughts (Besta et al., AAI 2024) generalizes further: thoughts form an arbitrary graph, not just a tree, allowing operations like *merging* two partial solutions or *refining* a thought using another thought as feedback — useful when a good solution is best assembled from combining several partial, individually-imperfect thoughts rather than following one branch to completion.

4.2 Where does ToT’s gain actually come from?

This is the single most important critical question for this topic (and appears in the seminar question bank for S1-1): **is the improvement coming from exploring more of the solution space, or from the quality of the state evaluator that decides what to prune?** A search procedure over a tree of bad-quality nodes with a perfect evaluator can still find the right answer by brute force; a sophisticated tree structure with a poor evaluator will prune good solutions early and never recover. In practice, ToT’s evaluator is *also an LLM call* — so the “state evaluator” is itself only as reliable as the same underlying model being asked a different question (“is this partial solution promising?”). This means ToT’s real cost-benefit trade-off is: spend $O(\text{branches} \times \text{depth})$ extra LLM calls, in exchange for a search structure that can recover from a bad first guess — but the ceiling on how well it can recover is set by how good the model already is at *judging* partial solutions, which is not obviously easier than solving the problem directly.

LATS (Language Agent Tree Search, Zhou et al., ICML 2024), assigned as extension reading in S1-1, combines this tree search with actual environment interaction (Monte Carlo Tree Search over ReAct-style trajectories, using real observations rather than only LLM-judged simulation) — worth previewing here as the “what if the evaluator were partly external/grounded rather than purely the same model judging itself” answer to the concern above.

4.3 Cost/accuracy summary across §1–4

Technique	Structure	Feedback signal required	Cost vs. single call	Best suited when
CoT	Linear chain	None	$\sim 1 \times$ (one longer generation)	Task benefits from explicit stepwise decomposition
Self-consistency	N independent chains, vote	None	$N \times$	Errors are uncorrelated across samples
ReAct	Chain + external actions	Tool observations (real, not judged)	$\sim 1 \times$ per step, k steps	Task needs external information/actions
Reflexion	Chain(s) + reflection across trials	An evaluator — ideally external ground truth	$\text{Trials} \times (1 \text{ chain} + 1 \text{ reflection})$	A cheap, reliable success/failure signal exists

Technique	Structure	Feedback signal required	Cost vs. single call	Best suited when
Tree/Graph-of-Thoughts	Search over branching thoughts	An LLM-judged state evaluator (internal)	branches $\times$ depth	Problem decomposes into evaluable partial solutions
LATS	MCTS over ReAct trajectories	Real observations + value estimates	highest of this table	High-stakes tasks where grounded evaluation is available

5. Planning: task decomposition, hierarchical planning, and world models

5.1 Why “planning” is a distinct claim from “reasoning”

In classical AI, **planning** has a precise meaning: given a formal model of the world (states, actions, preconditions, effects — e.g., PDDL), find a sequence of actions that provably transforms an initial state into a goal state, often with guarantees (soundness, completeness, sometimes optimality). This is a much stronger claim than “the model produced a plausible-looking numbered list of steps.” When a paper says an LLM “plans,” ask whether it means the classical sense (provable, verifiable against a formal model) or the colloquial sense (generated a sequential decomposition that is not verified against anything).

Useful patterns in the literature, roughly in order of how much of the “planning” is delegated to a verifiable external system versus the LLM alone:

- **Task decomposition** — the LLM breaks a goal into an ordered or partially-ordered list of sub-tasks in natural language, with no formal verification (e.g., a plain “first do X, then Y, then Z” prompt). This is the weakest and most common form, and is exactly what most “planning” agents in industry actually do.
- **Plan-and-execute** (Wang et al., “Plan-and-Solve Prompting,” ACL 2023) — explicitly separate a planning LLM call (produce the full sub-task list up front) from execution LLM calls (carry out each sub-task), rather than interleaving them as ReAct does. This trades ReAct’s ability to adapt the plan step-by-step for a clearer separation of concerns and the chance to *review* a plan before execution.
- **Hierarchical planning** — decompose recursively: a high-level plan’s steps are themselves sub-goals that get their own (possibly LLM-generated) sub-plans, bottoming out at directly executable actions. Useful when a task’s natural structure is itself hierarchical (e.g., “write a report” $\rightarrow$ “research”, “draft”, “revise”, each with their own steps).
- **LLM+P** (LLM-to-Planner) — use the LLM only to *translate* a natural-language task into a formal PDDL problem description, then hand the actual search off to a classical, sound-and-complete planner, and translate the resulting formal plan back into natural language. Here the LLM’s role is narrow (translation) and the planning guarantee comes entirely from the classical planner — a very different reliability profile than an LLM “planning” unaided.
- **World models** — rather than reasoning purely in language, learn or use an explicit predictive model of how the environment’s state changes in response to actions, and search or reason over that model’s predicted future states. This is the least mature of these patterns for language agents and the most actively researched.

5.2 The central objection: “LLMs can’t plan, but can help planning”

Kambhampati et al. (ICML 2024, “LLM-Modulo” frameworks) make an empirically-grounded argument that LLMs, used alone, are unreliable at classical planning tasks — they cannot verify their own plans against formal correctness criteria, and their apparent “planning” performance on many benchmarks degrades sharply on harder instances or adversarial rephrasing. Their proposed alternative, the **LLM-Modulo** framework, keeps the LLM in an *idea-generation* role (proposing candidate plans, translating between natural language and formal representations) while routing the actual verification and refinement through external, sound critics (a mix of classical planners, simulators, and formal verifiers) — structurally similar in spirit to LLM+P above, but framed as a general architecture rather than one specific pipeline.

The position to take away, and to defend or challenge in discussion: “planning” claims about LLMs split cleanly into (a) claims that are actually about *task decomposition* — a real and useful capability, but with no correctness guarantee, and (b) claims that are actually about *classical planning* — which current evidence suggests LLMs alone do not do reliably, and which should be verified against, or delegated to, an external formal system whenever failure is costly. When you encounter “planning” in a paper (or design a “planning module” for your project), state explicitly which of these two you mean.

6. In-class demo (instructor-led)

Run the same small task (e.g., a Game-of-24-style arithmetic puzzle, or a short multi-hop question) under three conditions, with token and wall-clock cost logged for each:

1. Plain CoT, single sample.
2. ReAct with access to a calculator/search tool.
3. Tree-of-Thoughts with a small branching factor (e.g., $b=3$, $\text{depth}=3$).

Discuss as a class: which condition got the right answer? At what token/latency cost? Which failure mode (if any) was visible in the wrong conditions’ traces — and does the accuracy gain of the more expensive condition justify its cost for *this specific task*, or would a simpler technique plus a cheap external verifier have been better? This exercise is deliberately designed to make the cost column of the §4.3 table concrete before you have to make the same trade-off for your own project.

7. Discussion questions for this session

1. For a task in your own project idea (from the Week 2 1-pager), which row of the §4.3 table fits best, and what would make you switch to a different row after seeing real results?
2. Reflexion’s feedback signal (§3.2) — for your project, what will evaluator actually be, and which of the three cases (external ground-truth / external heuristic / no external signal) does it fall into? If it’s case 3, what would you need to add to move it to case 1 or 2?
3. Kambhampati’s objection targets “planning” specifically — does the same objection apply to Reflexion’s “reflection,” or to ToT’s “evaluation”? What would distinguish a reliable self-assessment from an unreliable one in each case?
4. Self-consistency (§1.2) spends $n \times \text{compute}$ for a vote. Tree-of-Thoughts spends $\text{branches} \times \text{depth}$ compute for a search. Under what condition would you expect self-consistency to *beat* ToT despite being structurally simpler?

Further reading

- Wei et al. — *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models* (NeurIPS 2022).

- Yao et al. — *ReAct: Synergizing Reasoning and Acting in Language Models* (ICLR 2023).
- Shinn et al. — *Reflexion: Language Agents with Verbal Reinforcement Learning* (NeurIPS 2023).
- Yao et al. — *Tree of Thoughts: Deliberate Problem Solving with Large Language Models* (NeurIPS 2023).
- Besta et al. — *Graph of Thoughts: Solving Elaborate Problems with Large Language Models* (AAAI 2024).
- Zhou et al. — *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models* (LATS, ICML 2024).
- Wang et al. — *Plan-and-Solve Prompting* (ACL 2023).
- Kambhampati et al. — *LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks* (ICML 2024).
- Huang et al. — *Large Language Models Cannot Self-Correct Reasoning Yet* (ICLR 2024) — the direct challenge to §3.2's case-3 self-correction claims.